

Intro to Deep Learning

Lecture 12: Fine-Tuning Transformers for Classification

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. Why are encoder models useful for text classification?
2. How do we turn text into a batch for a BERT-like encoder?
3. What does a sequence-classification head do?
4. What happens during fine-tuning?
5. Why do we look beyond accuracy?
6. What practical choices matter most?

Lecture rhythm: encoder classifier -> tokenization/padding -> fine-tuning -> Trainer -> errors

Recap From Lecture 11

Last time:

```
raw text -> tokenizer -> input_ids + attention_mask -> transformer
```

We compared:

```
BERT-style encoders: masked prediction, bidirectional context  
GPT-style decoders: next-token prediction, generation
```

Today:

```
fine-tune an encoder for supervised classification
```

Part 1

Classification with encoders

BERT Pretraining Tasks

Original BERT used two self-supervised tasks.

Masked Language Modeling:

hide selected tokens and predict the original token

Next Sentence Prediction:

given sentence A and B, predict whether B followed A

Later models changed this recipe; for example, RoBERTa removed NSP and trained longer on more data.

Masked LM Details

Original BERT selects about 15% of tokens for prediction.

For selected tokens:

80% -> replace with [MASK]
10% -> replace with random token
10% -> keep unchanged

This avoids making pretraining rely only on seeing artificial [MASK] tokens.

Next Sentence Prediction

NSP turns two text segments into a binary classification task.

Example positive pair:

```
A: The team won the final.  
B: Fans celebrated in the streets.  
label: IsNext
```

Example negative pair:

```
A: The team won the final.  
B: The recipe calls for fresh basil.  
label: NotNext
```

Original BERT used NSP to encourage sentence-pair understanding.

Text Classification

Text classification asks:

```
given a text, predict one label
```

Examples:

- sentiment classification
- spam detection
- topic classification
- intent detection
- toxicity detection

A running example is AG News topic classification.

AG News

AG News is a 4-class news classification dataset.

Typical labels:

```
World  
Sports  
Business  
Sci/Tech
```

Input:

```
news headline or short article
```

Output:

```
one of four class labels
```

Why Encoder Models Fit Classification

For classification, the model should read the whole input.

BERT-style encoders are useful because:

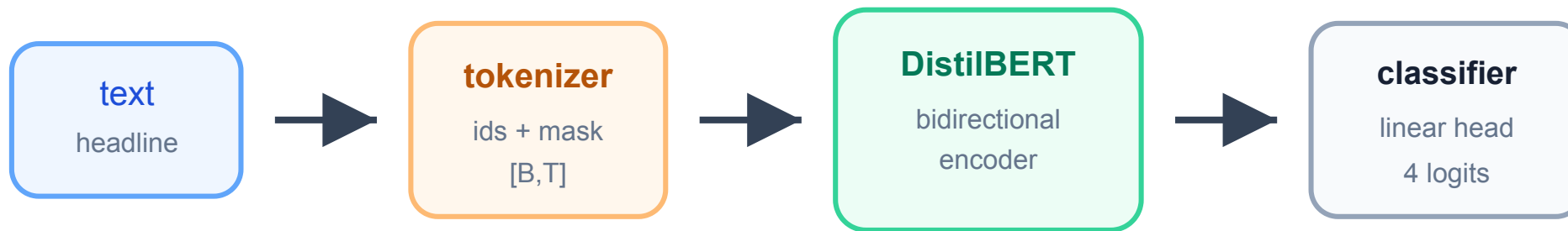
each token can use left and right context

The model builds a representation of the whole sequence.

Then a classification head maps that representation to logits.

Encoder Classification Pipeline

Fine-tuning turns a pretrained encoder into a classifier



Supervised labels update the head and, in full fine-tuning, the encoder too.

From Pretraining To Fine-Tuning

Pretraining:

```
learn general language representations
```

Fine-tuning:

```
update the pretrained model on labeled task data
```

For AG News:

```
text + label -> update encoder + classifier head
```

The model adapts general language features to news topics.

Sequence Classification Head

The encoder produces hidden states.

For classification, we need:

```
one vector for the whole input
```

Then:

```
vector -> linear layer -> class logits
```

For AG News:

```
logits shape = [B, 4]
```

Sequence Representation Choices

BERT-like encoders produce one vector per token.

For classification, we need one vector for the whole text.

Common choices:

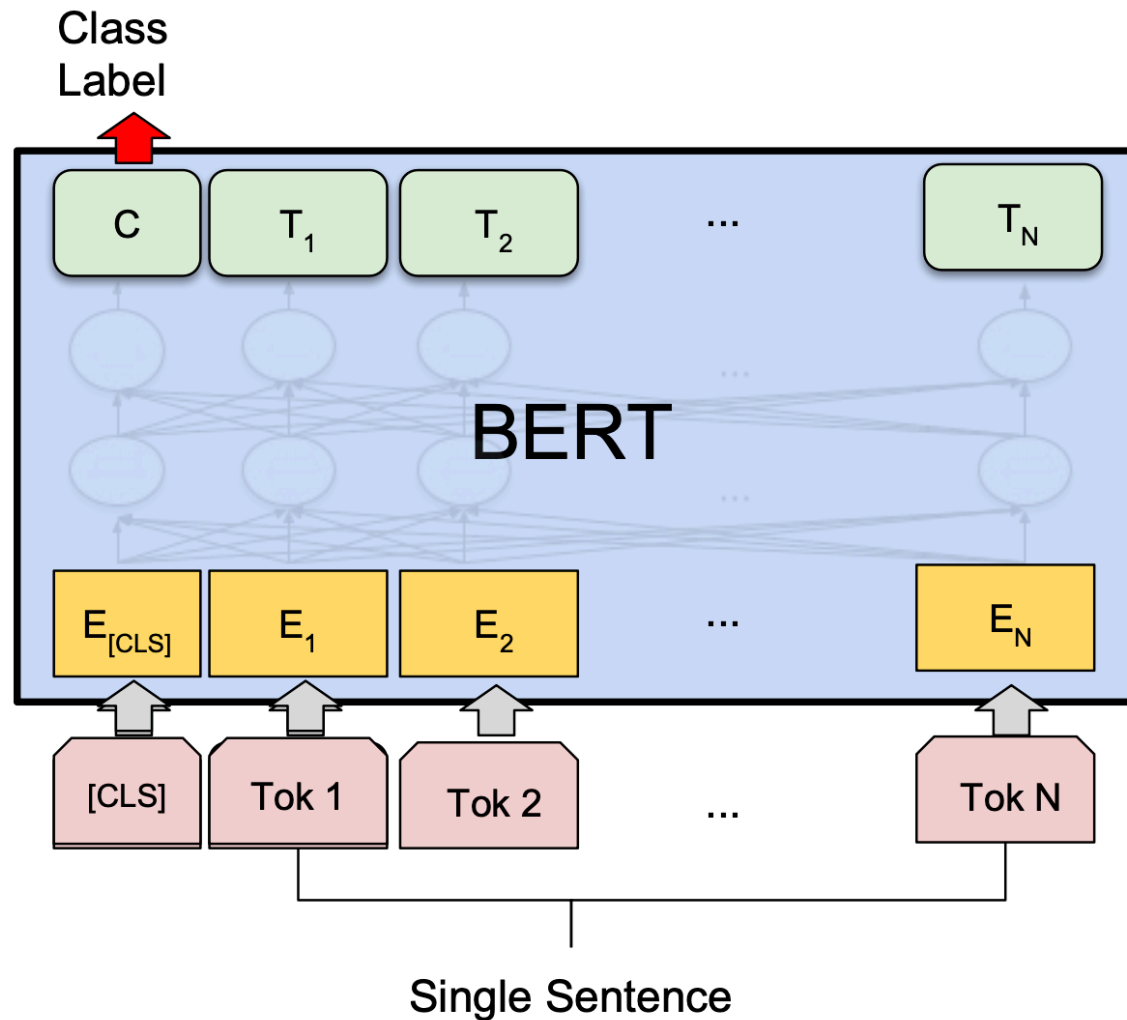
```
[CLS] / first-token representation  
mean pooling over token representations  
task-specific pooling layer
```

Exact implementation details can differ between BERT-like models.

Important idea:

```
many token vectors -> one sequence vector -> logits
```

BERT Overview



Token Representations To Logits

Classification compresses token states into one sequence vector



Why Pooling Matters

The sequence vector is the bottleneck before classification.

It should contain the information needed for the label.

For short news texts:

```
topic signal is often spread across the headline
```

For longer texts:

```
important clues may appear late in the document
```

Pooling and truncation interact.

Part 2

Tokenization, truncation, and padding

Tokenization Pipeline

Raw examples contain text strings.

The tokenizer creates:

```
input_ids  
attention_mask
```

Typical code:

```
tokenizer(  
    batch["text"],  
    truncation=True,  
    max_length=...  
)
```

This is the preprocessing step before training.

Mini-Check: Long Text Error

Validation errors cluster on long articles.

The headline looks neutral, but the final paragraph contains the topic clue.

Current preprocessing:

```
tokenizer(text, truncation=True, max_length=64)
```

What would you inspect before changing the model?

Answer

Inspect what text remains after tokenization and truncation.

The model may never see the final paragraph if:

```
tokenizer(text, truncation=True, max_length=64)
```

keeps only the beginning.

Before changing the model, check:

- tokenized length distribution
- examples after decoding truncated `input_ids`
- whether label evidence appears late in the text

Truncation Is A Modeling Decision

Models have a maximum sequence length.

Truncation means:

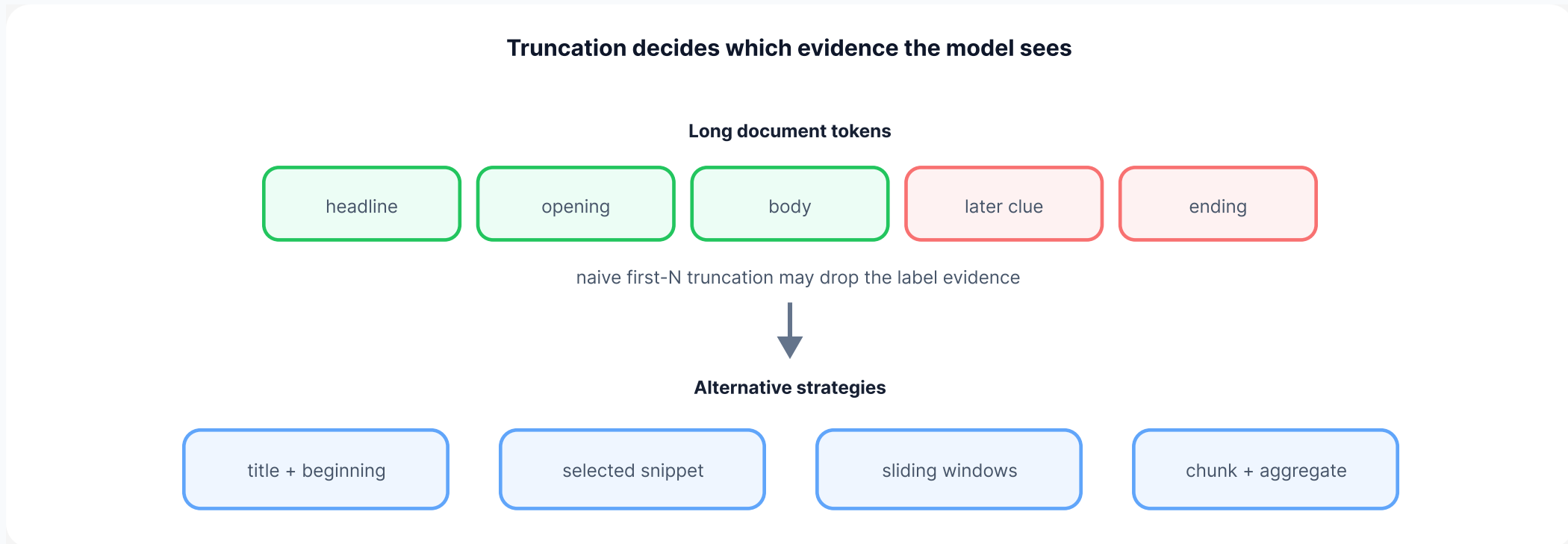
```
cut text that is too long
```

Why it matters:

- controls memory use
- controls training speed
- may remove useful information

For text classification, truncation can change the example the model sees.

Truncation Strategies



For short texts, first-N truncation is often fine.

For long documents, the right strategy depends on where the label evidence appears.

Padding

Batches need tensors of equal length.

Padding means:

```
add pad tokens to shorter examples
```

Attention mask tells the model:

```
1 = real token  
0 = padding token
```

So padding does not become meaningful text.

Dynamic Padding

Instead of padding every example to a fixed global length, we can pad per batch.

Hugging Face:

```
DataCollatorWithPadding(tokenizer=tokenizer)
```

This pads each batch only as much as needed.

Benefits:

- less wasted computation
- simpler batching
- fewer shape headaches

Padding And Attention Mask

Padding makes equal-length batches; the mask hides padding

tokens

Sports

team

[PAD]

[PAD]

[PAD]

[PAD]

mask

1

1

0

0

0

0

tokens

Market

rises

after

report

was

published

mask

1

1

1

1

1

1

1 = real token, 0 = padding token

Fine-Tuning Objective

For multiclass classification, we use cross-entropy.

For one example:

```
logits -> softmax probabilities  
loss = -log probability of correct class
```

In PyTorch/Hugging Face, we pass:

```
logits and integer class labels
```

If labels are passed to the model, the forward pass returns cross-entropy loss and logits.

Part 3

Fine-tuning mechanics

What Gets Updated?

During full fine-tuning:

```
classifier head weights update  
encoder weights also update
```

This is different from frozen feature extraction.

Benefits:

```
model adapts to the target task
```

Risk:

```
overfitting or damaging useful pretrained features
```

Learning Rate Is Small

Transformer fine-tuning usually uses a small learning rate.

Typical scale:

`1e-5 to 5e-5`

Why?

`pretrained weights are already useful
we want controlled adaptation`

Too high LR can make fine-tuning unstable.

Batch Size And Memory

Transformer memory depends on:

```
batch size  
sequence length  
model size
```

If training runs out of memory:

- reduce batch size
- reduce max length
- use a smaller model
- use gradient accumulation if needed

In this course, we keep subsets and models practical.

Part 4

Trainer workflow

Hugging Face Trainer

The `Trainer` API is a common high-level fine-tuning loop.

It collects the main training pieces:

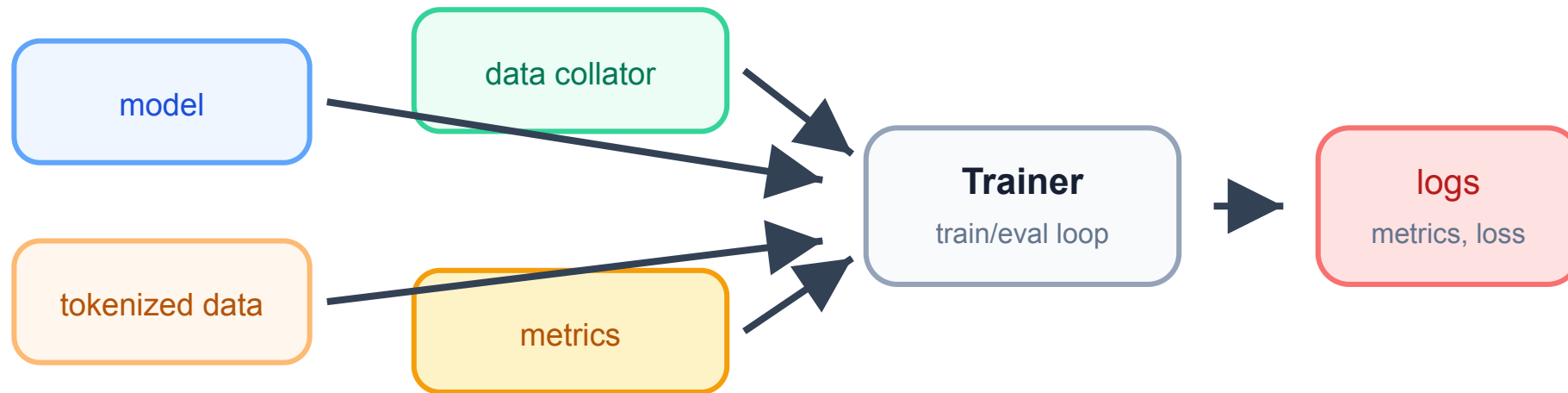
- model
- training arguments
- train dataset
- validation dataset
- data collator
- metrics

The core learning idea is still:

```
forward -> loss -> backward -> optimizer step
```

Trainer Inputs

Trainer wraps the same training loop pieces



The Head Starts From Scratch

The encoder starts from pretrained weights.

The classification head usually starts randomly initialized.

So early in fine-tuning:

```
head learns task mapping quickly  
encoder adapts more slowly
```

This is one reason small learning rates and short training runs are common.

Fine-Tuning Regimes

There are several ways to adapt a pretrained encoder.

Head-only:

```
freeze encoder, train classifier head
```

Partial fine-tuning:

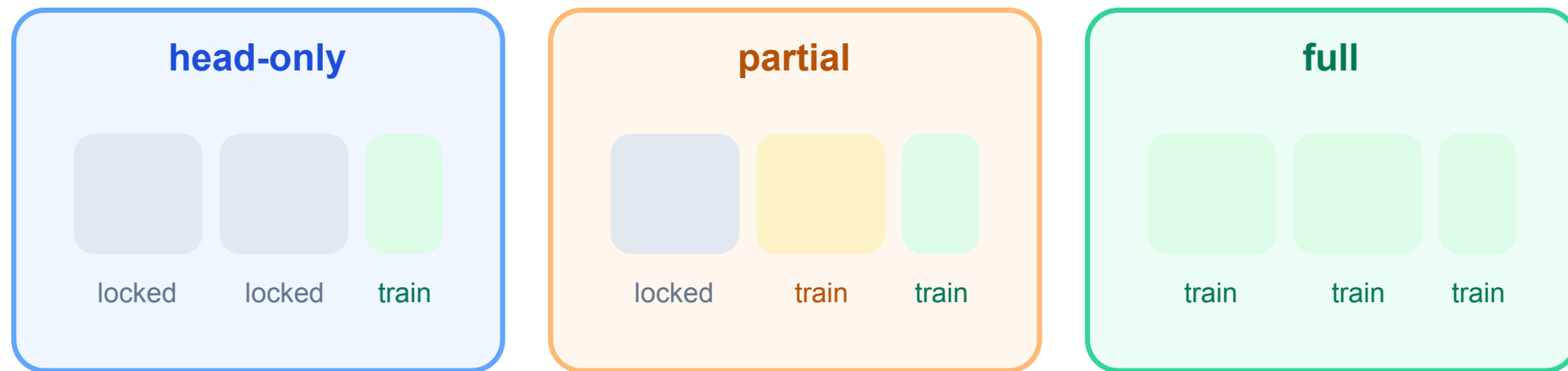
```
freeze lower layers, train upper layers + head
```

Full fine-tuning:

```
train all encoder layers + head
```

Fine-Tuning Regimes: Picture

Fine-tuning regimes differ by which weights update



Which Regime To Choose?

Head-only is useful when:

very little data, limited compute, quick baseline

Partial fine-tuning is useful when:

some domain shift, but overfitting risk is high

Full fine-tuning is useful when:

enough labeled data and the task needs adaptation

There is no universally best choice.

Small-Data Behavior

Pretraining helps, but small labeled datasets are still fragile.

Typical behavior:

```
fast improvement in first epochs  
possible overfitting soon after  
high variance between runs
```

Why?

- model is powerful
- labels may be limited
- examples may be ambiguous

Small validation sets can also be noisy.

What More Labels Change

More labeled data usually helps by:

- reducing overfitting
- covering more wording patterns
- making validation estimates more stable
- giving the encoder safer adaptation signal

Small Data And Fine-Tuning Regime

With very little labeled data:

head-only or partial fine-tuning may be safer

With enough labeled data:

full fine-tuning can adapt representations better

Practical question:

do we need to adapt the whole encoder,
or only learn a task-specific head?

The answer is empirical.

Baselines Still Matter

Transformers are strong, but simple baselines can be useful.

Example optional baseline:

```
TF-IDF + logistic regression
```

Why compare?

- detect overengineering
- estimate task difficulty
- catch data leakage
- understand what the transformer adds

Strong engineering starts with fair baselines.

BERT Family: Important Variants

Model	Main idea
BERT	Original encoder trained with MLM + NSP
RoBERTa	Stronger training recipe; no NSP
DistilBERT	Smaller/faster student distilled from BERT
ALBERT	Parameter sharing and factorized embeddings
DeBERTa	Improved content/position attention
MiniLM	Compact model distilled from transformer behavior

The shared theme: keep the encoder idea, change training recipe, architecture, or compression.

Model Distillation

Distillation: train a smaller model to imitate a stronger teacher



DistilBERT is a compressed BERT-like encoder, not a different task family.

Distillation is useful when a large model works well but is too expensive to deploy.

The student is trained to imitate the teacher while staying smaller and faster.

Why Distillation Works

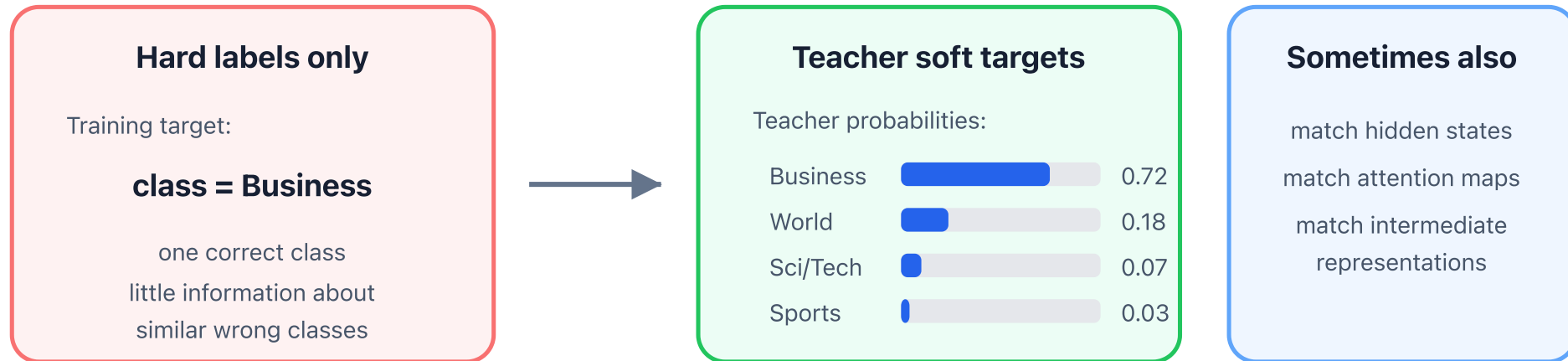
Hard labels say only which class is correct.

Teacher probabilities show softer structure:

which wrong classes are almost plausible?
which examples are easy or uncertain?

Some distillation methods also train the student to match teacher hidden states or attention maps.

Why distillation can beat training a small model from labels only



Distillation transfers behavior, not just labels.

Part 5

Evaluation and error analysis

Practical Decisions

Important fine-tuning choices:

- model checkpoint
- max sequence length
- batch size
- learning rate
- number of epochs
- train/validation subset size
- metric selection

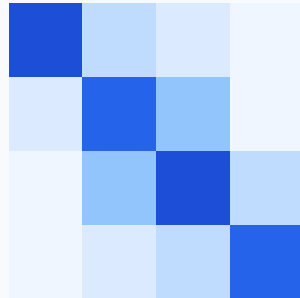
Small changes can matter.

Document them when comparing runs.

Confusion Matrix And Mistakes

Accuracy hides which classes are confused

confusion matrix



inspect mistakes

Sports misread as Business?
Sci/Tech confused with Business?
Ambiguous headline or truncation?

Common Mistakes

Mistake 1:

```
tokenizer and model checkpoint do not match
```

Mistake 2:

```
forgetting truncation for long texts
```

Mistake 3:

```
evaluating only accuracy
```

Mistake 4:

```
not reading misclassified examples
```

Debugging Checklist

Before training, check:

- label names match label IDs
- tokenized example has `input_ids` and `attention_mask`
- model has `num_labels = 4`
- logits shape is `[B, 4]`
- metrics use predicted class IDs
- validation set is not used for training
- confusion matrix labels are ordered correctly

Practical Workflow

A typical fine-tuning workflow:

1. inspect labels and examples
2. tokenize text and check truncation
3. use dynamic padding
4. load an encoder for sequence classification
5. fine-tune with a small learning rate
6. evaluate beyond accuracy
7. inspect confusion matrix and mistakes

Takeaways

Fine-tuning adapts pretrained encoders to labeled tasks.

Key ideas:

- tokenizer output becomes model input tensors
- classification head maps sequence representation to logits
- small LR is important for pretrained weights
- accuracy alone is not enough
- confusion matrices and mistakes explain model behavior

Next lecture:

LLM adaptation: SFT, LoRA, and RLHF overview